Федеральное государственное образовательное бюджетное учреждение
высшего образования

«ФИНАНСОВЫЙ УНИВЕРСИТЕТ ПРИ ПРАВИТЕЛЬСТВЕ РФ»
Департамент анализа данных и машинного обучения

Отчет по контрольной работе

на тему: «Разработка консольного музыкального плеера с управлением плейлистом на языке С»

по дисциплине «Низкоуровневое программирование»

Студента группы ПМ23-1
Факультета информационных технологий и анализа больших данных

Тищенко И.С.

Преподаватель
Крахмалев О.Н.

Москва 2026

Описание программы

Оглавление

Аннотация....2

1. Общие сведения....3

1.1. Наименование программы....3

1.2. Назначение описания....3

1.3. Программное обеспечение....3

1.4. Язык программирования....3

2. Функциональное назначение....4

3. Описание логической структуры....6

3.1. Общий алгоритм....6

3.2. Структуры данных....7

3.3. Глобальное состояние....8

4. Описание функций файла main.c....9

4.1. Служебные функции строк и интерфейса....9

4.2. Синхронизация потоков....13

4.3. Кодировки и проверка файлов....16

4.4. Настройки и громкость....17

4.5. Воспроизведение и MCI....21

4.6. Управление двусвязным списком....26

4.7. Импорт и редактирование плейлиста....28

4.8. Сохранение и загрузка данных....32

4.9. Чтение метаданных ID3v1.....33
4.10. Главная функция программы.....34
5. Описание функций файла tests/test_musicplayer.c.....36
5.1. Макрос TEST_ASSERT.....36
5.1. Вспомогательные функции тестового модуля.....36
5.2. Тестовые сценарии.....39
5.3. Главная функция тестового модуля.....44
6. Используемые технические средства.....45
7. Вызов и загрузка.....46
8. Входные данные.....47
9. Выходные данные.....48
10. Файлы кода.....49
11. Контроль полноты описания.....50
Приложение А. Полный листинг файла main.c.....51
Приложение Б. Полный листинг файла tests/test_musicplayer.c.....76

Аннотация

Консольный MP3-плеер разработан на языке C для операционной системы Windows. Описание дополнено подробным разбором каждой функции исходного кода.

Рассмотрены рабочие функции файла `main.c` и функции автоматизированного тестового модуля `tests/test_musicplayer.c`. Для каждой функции приведены сигнатура, назначение, параметры, возвращаемое значение, алгоритм, побочные эффекты, особые условия и основные связанные вызовы.

1. Общие сведения

1.1. Наименование программы

Полное наименование: «Разработка консольного MP3-плеера с управлением плейлистом и воспроизведением через Windows API на языке C».

Краткое наименование: `MusicPlayer`.

1.2. Назначение описания

Описание предназначено для изучения структуры программы, сопровождения исходного кода и проверки соответствия реализации заявленным функциям.

1.3. Программное обеспечение

Для сборки и выполнения программы требуются:

- операционная система Windows;
- компилятор C с поддержкой WinAPI, например Microsoft Visual C;
- CMake версии 3.10 или новее;
- системная библиотека `winmm`;
- консоль с поддержкой UTF-8.

1.4. Язык программирования

Основная программа и тестовый модуль написаны на языке C. Для мультимедийных операций, работы с файлами Unicode и потоками используются функции Windows API.

2. Функциональное назначение

Программа предназначена для локального воспроизведения MP3-файлов и управления плейлистом из консоли. Реализованы:

- добавление одного трека и импорт MP3-файлов из папки;
- хранение треков в двусвязном списке;
- воспроизведение, остановка, пауза, продолжение и циклическая навигация;
- отображение длительности трека;
- управление громкостью через команды `+` и `-`;
- сортировка, поиск и перемешивание плейлиста;
- сохранение плейлиста и громкости между запусками;
- чтение ID3v1-тегов;
- автоматический переход к следующему треку в фоновом потоке;
- автоматизированные тесты ключевой логики.

Функциональные ограничения:

- воспроизводятся только файлы `.mp3`;
- импорт папки не является рекурсивным;
- поддерживаются теги ID3v1, но не ID3v2;
- воспроизведение зависит от возможностей MCI в установленной Windows-системе.

На рисунке 1 показано главное меню программы после запуска.

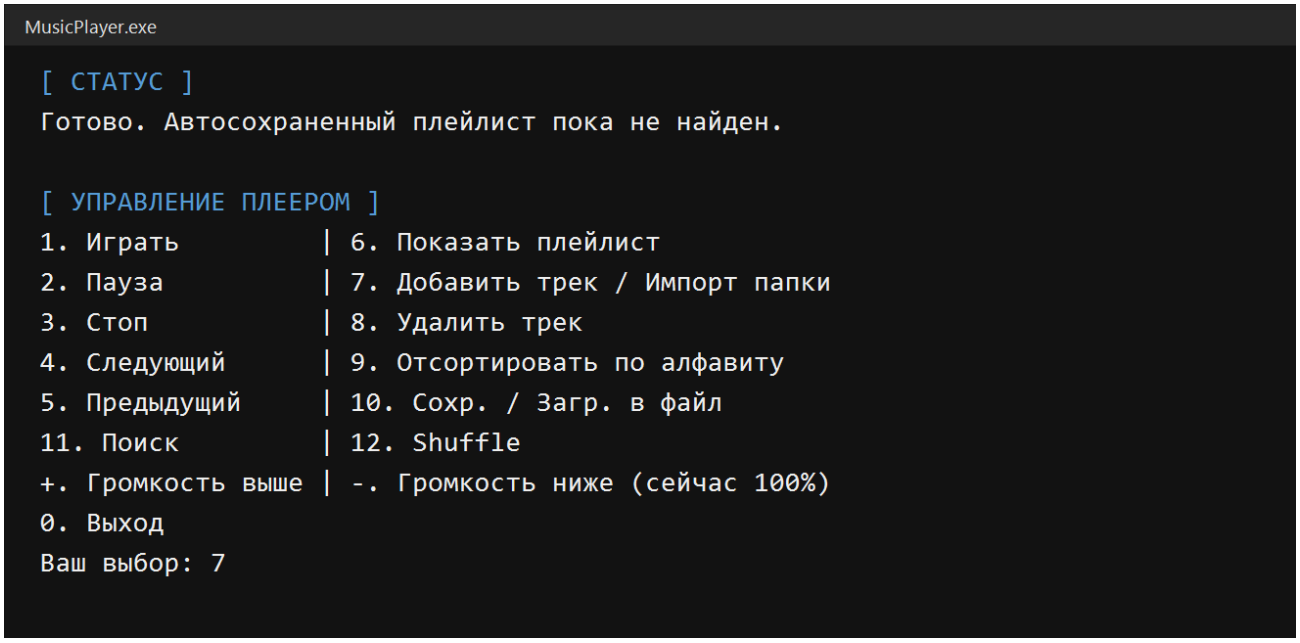


Рисунок 1 - Главное меню консольного плеера

3. Описание логической структуры

3.1. Общий алгоритм

При запуске программа настраивает UTF-8 консоль, инициализирует критическую секцию и пустой плейлист, загружает настройки и ранее сохраненный плейлист. Затем создается фоновый поток автоперехода. Основной поток циклически отображает меню, читает команду пользователя и вызывает соответствующую функцию. При выходе поток автоперехода завершается, плейлист и громкость сохраняются, MCI-устройство закрывается, память списка освобождается.

Общая схема взаимодействия компонентов приведена на рисунке 2.

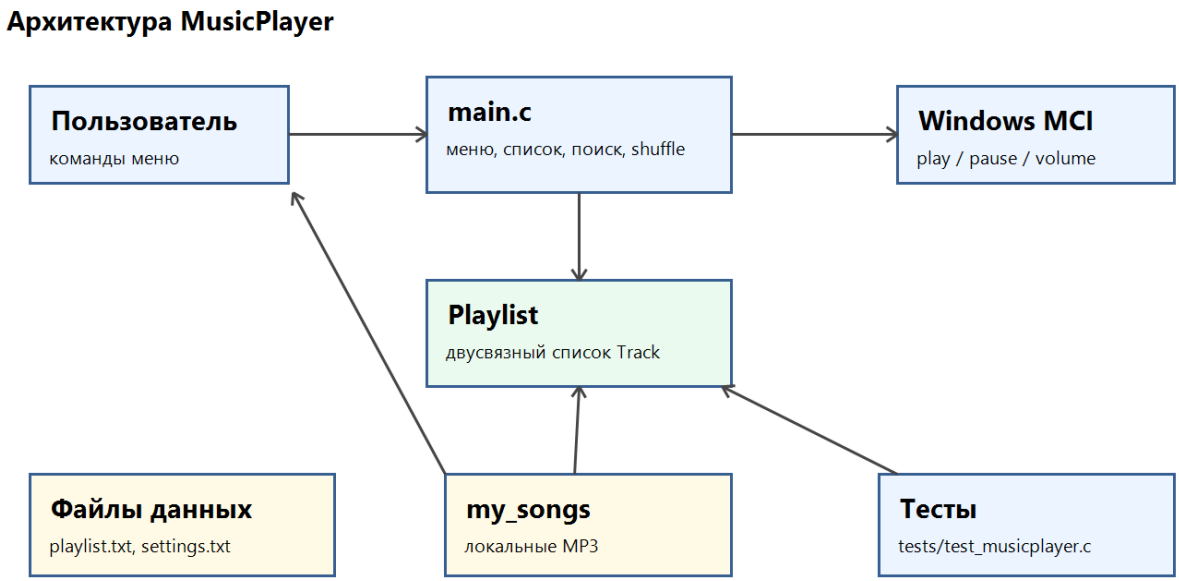


Рисунок 2 - Архитектура программы

3.2. Структуры данных

Узел `Track` хранит название, исполнителя, путь к файлу и ссылки `next` и `prev`. Структура `Playlist` хранит указатели `head`, `tail`, `current` и размер списка `size`.

На рисунке 3 приведены объявления основных структур данных.

main.c

```c
#define TITLE_SIZE 100
#define ARTIST_SIZE 100
#define PATH_SIZE 512

typedef struct Track {
    char title[TITLE_SIZE];
    char artist[ARTIST_SIZE];
    char filepath[PATH_SIZE];
    struct Track* next;
    struct Track* prev;
} Track;

typedef struct Playlist {
    Track* head;
    Track* tail;
    Track* current;
    int size;
} Playlist;
```

Рисунок 3 - Объявления структур Track и Playlist

На рисунке 4 показана организация двусвязного списка.

Двусвязный список плейлиста

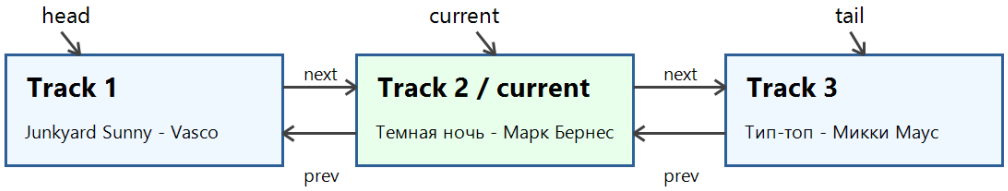


Рисунок 4 - Двусвязный список плейлиста

3.3. Глобальное состояние

Идентификатор	Назначение
`g_track_open`	Признак открытого MCI-устройства `track`
`g_is_paused`	Признак паузы
`g_volume`	Громкость по шкале MCI от 0 до 1000
`g_status_message`	Текущее сообщение интерфейса
`g_panel_text`	Панель результатов операций
`g_player_lock`	Критическая секция для общего состояния
`g_auto_next_thread`	Дескриптор фонового потока
`g_auto_next_running`	Атомарный флаг работы потока
`g_last_track_start_tick`	Время запуска трека для защитной задержки

4. Описание функций файла `main.c`

В основном исходном файле определено функций: **47**.

4.1. Служебные функции строк и интерфейса

4.1.1. Функция `copy_string`

Сигнатура

```
static void copy_string(char* destination, size_t destination_size, const
char* source);
```

Назначение. Безопасно копирует строку в буфер ограниченного размера.

Параметры. `destination` - выходной буфер; `destination_size` - размер буфера в байтах; `source` - исходная строка или `NULL`.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. При нулевом размере буфера завершает работу. Для `NULL`-источника записывает пустую строку. В остальных случаях вызывает `snprintf`, обеспечивая завершающий нулевой символ.

Побочные эффекты. Изменяет содержимое `destination`.

Ошибки и особые условия. Не сообщает об усечении строки отдельно: при недостаточном размере буфера строка копируется частично.

Основные связанные вызовы. `snprintf`.

4.1.2. Функция `strip_surrounding_quotes`

Сигнатура

```
static void strip_surrounding_quotes(char* value);
```

Назначение. Удаляет внешние двойные кавычки из пути, введенного пользователем.

Параметры. `value` - изменяемая строка.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет длину строки. Если кавычки находятся с обеих сторон, сдвигает полезную часть через `memmove`. Если кавычка есть только в начале, удаляет только ее.

Побочные эффекты. Изменяет строку `value` на месте.

Ошибки и особые условия. Ожидает корректный изменяемый буфер с завершающим нулевым символом.

Основные связанные вызовы. `strlen`, `memmove`.

4.1.3. Функция `set_status`

Сигнатура

```
static void set_status(const char* format, ...);
```

Назначение. Формирует текущее статусное сообщение консольного интерфейса.

Параметры. `format` - форматная строка; последующие аргументы соответствуют спецификаторам формата.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает список вариативных аргументов и записывает сформированный текст в глобальный буфер `g_status_message` через `vsnprintf`.

Побочные эффекты. Перезаписывает глобальную строку статуса.

Ошибки и особые условия. Длинное сообщение усекается по размеру `STATUS_MESSAGE_SIZE`.

Основные связанные вызовы. `va_start`, `vsnprintf`, `va_end`.

4.1.4. Функция `clear_panel`

Сигнатура

```
static void clear_panel(void);
```

Назначение. Очищает дополнительную текстовую панель результатов.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Записывает нулевой символ в начало глобального буфера `g_panel_text`.

Побочные эффекты. Очищает глобальную панель.

Ошибки и особые условия. Ошибочные ситуации отсутствуют.

Основные связанные вызовы. Внешние функции не вызывает.

4.1.5. Функция `append_panel`

Сигнатура

```
static void append_panel(const char* format, ...);
```

Назначение. Добавляет форматированный текст в панель результатов.

Параметры. `format` - форматная строка; последующие аргументы соответствуют спецификаторам формата.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Определяет текущую длину панели, вычисляет оставшийся объем и дописывает текст через `vsnprintf`, если в буфере есть место.

Побочные эффекты. Изменяет глобальный буфер `g_panel_text`.

Ошибки и особые условия. Если свободного места нет, новый фрагмент не добавляется. Длинный фрагмент может быть усечен.

Основные связанные вызовы. `strlen`, `va_start`, `vsnprintf`, `va_end`.

4.1.6. Функция `print_playlist`

Сигнатура

```
void print_playlist(Playlist* pl);
```

Назначение. Формирует текстовое представление плейлиста.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Очищает панель, проходит список от `head` к `tail`, нумерует треки и выделяет текущий трек маркерами. Затем обновляет статус.

Побочные эффекты. Изменяет панель результатов и строку статуса.

Ошибки и особые условия. Для пустого списка выводит сообщение и завершает работу.

Основные связанные вызовы. `clear_panel`, `append_panel`, `set_status`.

4.1.7. Функция `print_menu`

Сигнатура

```
void print_menu();
```

Назначение. Выводит список доступных команд консольного меню.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Печатает команды управления и текущую громкость в процентах.

Побочные эффекты. Пишет в стандартный вывод.

Ошибки и особые условия. Ошибочные ситуации отсутствуют.

Основные связанные вызовы. `printf`.

4.1.8. Функция `render_screen`

Сигнатура

```
void render_screen();
```

Назначение. Перерисовывает экран приложения.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Очищает консоль, печатает статус, при наличии текста выводит панель результатов и затем меню.

Побочные эффекты. Очищает экран и пишет в стандартный вывод.

Ошибки и особые условия. Зависит от доступности команды Windows `cls`.

Основные связанные вызовы. `system`, `printf`, `print_menu`.

4.2. Синхронизация потоков

4.2.9. Функция `lock_player`

Сигнатура

```
static void lock_player(void);
```

Назначение. Входит в критическую секцию плеера, если она уже инициализирована.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет флаг `g_player_lock_ready` и вызывает системную функцию входа в критическую секцию.

Побочные эффекты. Может блокировать вызывающий поток до освобождения критической секции.

Ошибки и особые условия. До инициализации критической секции функция намеренно ничего не делает.

Основные связанные вызовы. `EnterCriticalSection`.

4.2.10. Функция `unlock_player`

Сигнатура

```
static void unlock_player(void);
```

Назначение. Освобождает критическую секцию плеера, если она инициализирована.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет флаг `g_player_lock_ready` и вызывает системную функцию выхода из критической секции.

Побочные эффекты. Разрешает продолжить выполнение другому ожидающему потоку.

Ошибки и особые условия. Функцию следует вызывать только после соответствующего входа в критическую секцию.

Основные связанные вызовы. `LeaveCriticalSection`.

4.2.11. Функция `auto_next_monitor`

Сигнатура

```
DWORD WINAPI auto_next_monitor(LPVOID parameter);
```

Назначение. Является функцией фонового потока автоматического перехода к следующему треку.

Параметры. `parameter` - указатель `LPVOID`, фактически содержащий `Playlist*`.

Возвращаемое значение. `DWORD` со значением `0` после штатного завершения потока.

Алгоритм работы. Пока установлен атомарный флаг работы, засыпает на `AUTO_NEXT_POLL_MS`, входит в критическую секцию и проверяет режим MCI. Если после защитной задержки трек остановился естественным образом, запускает следующий.

Побочные эффекты. Работает в отдельном потоке, может менять статус и текущий трек, запускает звук.

Ошибки и особые условия. Ошибка запроса режима MCI игнорируется до следующей итерации. Доступ к общему состоянию защищен критической секцией.

Основные связанные вызовы. `InterlockedCompareExchange`, `Sleep`, `lock_player`, `GetTickCount`, `mciSendStringA`, `set_status`, `play_next`, `unlock_player`.

4.2.12. Функция `start_auto_next_monitor`

Сигнатура

```
static void start_auto_next_monitor(Playlist* pl);
```

Назначение. Запускает фоновый поток автоматического перехода.

Параметры. `pl` - плейлист, адрес которого передается фоновому потоку.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Устанавливает атомарный флаг работы и создает поток с точкой входа `auto_next_monitor`.

Побочные эффекты. Может создать системный поток и изменить глобальный дескриптор.

Ошибки и особые условия. При ошибке `CreateThread` сбрасывает флаг и выводит сообщение.

Основные связанные вызовы. `InterlockedExchange`, `CreateThread`, `set_status`.

4.2.13. Функция `stop_auto_next_monitor`

Сигнатура

```
static void stop_auto_next_monitor(void);
```

Назначение. Корректно завершает фоновый поток автоперехода.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Если поток создан, сбрасывает атомарный флаг, ожидает завершения, закрывает дескриптор и обнуляет его.

Побочные эффекты. Останавливает поток и освобождает системный дескриптор.

Ошибки и особые условия. При отсутствии потока ничего не делает.

Основные связанные вызовы. `InterlockedExchange`, `WaitForSingleObject`, `CloseHandle`.

4.3. Кодировки и проверка файлов

4.3.14. Функция `utf8_to_wide`

Сигнатура

```
static int utf8_to_wide(const char* source, wchar_t* destination, int
destination_size);
```

Назначение. Преобразует строку UTF-8 в wide-строку Windows.

Параметры. `source` - исходная UTF-8 строка; `destination` - выходной буфер `wchar_t`; `destination_size` - число элементов выходного буфера.

Возвращаемое значение. `1` при успешном преобразовании, иначе `0`.

Алгоритм работы. Проверяет указатели и размер буфера, затем вызывает `MultiByteToWideChar` с кодовой страницей `CP_UTF8`.

Побочные эффекты. Записывает wide-строку в `destination`.

Ошибки и особые условия. Возвращает `0` при `NULL`, неположительном размере, ошибочной UTF-8 строке или недостаточном выходном буфере.

Основные связанные вызовы. `MultiByteToWideChar`.

4.3.15. Функция `file_exists_utf8`

Сигнатура

```
static int file_exists_utf8(const char* filepath);
```

Назначение. Проверяет наличие обычного файла по UTF-8 пути.

Параметры. `filepath` - путь к файлу в UTF-8.

Возвращаемое значение. `1`, если файл существует и не является каталогом; иначе `0`.

Алгоритм работы. Преобразует путь в wide-строку, получает атрибуты через WinAPI и исключает каталоги.

Побочные эффекты. Не изменяет состояние программы.

Ошибки и особые условия. Возвращает `0`, если путь невозможно преобразовать или `GetFileAttributesW` не находит объект.

Основные связанные вызовы. `utf8_to_wide`, `GetFileAttributesW`.

4.3.16. Функция `contains_ignore_case_utf8`

Сигнатура

```
static int contains_ignore_case_utf8(const char* text, const char* query);
```

Назначение. Проверяет наличие подстроки без учета регистра, включая строки с кириллицей.

Параметры. `text` - проверяемая строка UTF-8; `query` - искомая непустая подстрока UTF-8.

Возвращаемое значение. `1` при наличии совпадения, иначе `0`.

Алгоритм работы. Преобразует обе строки в wide-формат. Затем последовательно сравнивает фрагменты текста через `CompareStringOrdinal` с флагом игнорирования регистра. Если преобразование невозможно, использует резервный регистрозависимый `strstr`.

Побочные эффекты. Не изменяет состояние программы.

Ошибки и особые условия. Пустой запрос и `NULL`-указатели приводят к возврату `0`. При ошибке Unicode-преобразования резервный поиск чувствителен к регистру.

Основные связанные вызовы. `utf8_to_wide`, `wcslen`, `CompareStringOrdinal`, `strstr`.

4.4. Настройки и громкость

4.4.17. Функция `save_settings`

Сигнатура

```
static void save_settings(void);
```

Назначение. Сохраняет текущий уровень громкости в текстовый файл настроек.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Открывает служебное хранилище настроек на запись, записывает строку `volume=<значение>` и закрывает файл.

Побочные эффекты. Создает или перезаписывает файл служебное хранилище настроек.

Ошибки и особые условия. Если файл невозможно открыть, завершает работу без изменения статуса интерфейса.

Основные связанные вызовы. `fopen`, `fprintf`, `fclose`.

4.4.18. Функция `load_settings`

Сигнатура

```
static void load_settings(void);
```

Назначение. Загружает сохраненный уровень громкости.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Читает служебное хранилище настроек построчно. Для строк формата `volume=<число>` применяет ограничение диапазона и обновляет глобальную громкость.

Побочные эффекты. Может изменить `g_volume`.

Ошибки и особые условия. Отсутствие файла считается допустимым состоянием. Некорректные строки пропускаются.

Основные связанные вызовы. `fopen`, `fgets`, `sscanf`, `clamp_volume`, `fclose`.

4.4.19. Функция `clamp_volume`

Сигнатура

```
static int clamp_volume(int volume);
```

Назначение. Ограничивает уровень громкости допустимым диапазоном MCI-плеера.

Параметры. `volume` - проверяемое целое значение.

Возвращаемое значение. Значение в диапазоне от `VOLUME_MIN` до `VOLUME_MAX` включительно.

Алгоритм работы. Сравнивает аргумент с нижней и верхней границами. Значения вне диапазона заменяет ближайшей границей.

Побочные эффекты. Не изменяет состояние программы.

Ошибки и особые условия. Ошибочные ситуации отсутствуют.

Основные связанные вызовы. Внешние функции не вызывает.

4.4.20. Функция `apply_volume_to_open_track`

Сигнатура

```
static MCIERROR apply_volume_to_open_track(void);
```

Назначение. Применяет глобальный уровень громкости к открытому MCI-устройству с псевдонимом `track`.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Код `MCIERROR`: `0` при успехе или код ошибки.

Алгоритм работы. Формирует MCI-команду `setaudio track volume to ...` и отправляет ее в MCI.

Побочные эффекты. Изменяет громкость открытого аудиоустройства.

Ошибки и особые условия. Если устройство не открыто или команда не поддерживается, возвращает код ошибки MCI.

Основные связанные вызовы. `snprintf`, `mciSendStringA`.

4.4.21. Функция `print_volume`

Сигнатура

```
static void print_volume(void);
```

Назначение. Выводит текущую громкость в строку статуса интерфейса.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Очищает панель результатов и переводит шкалу MCI 0-1000 в проценты делением на 10.

Побочные эффекты. Очищает панель и изменяет статус.

Ошибки и особые условия. Ошибочные ситуации отсутствуют.

Основные связанные вызовы. `clear_panel`, `set_status`.

4.4.22. Функция `change_volume`

Сигнатура

```c
static void change_volume(int delta);
```

Назначение. Изменяет громкость на заданную величину и сохраняет настройку.

Параметры. `delta` - положительное или отрицательное изменение шкалы MCI.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Запоминает старое значение, вычисляет новое через `clamp_volume`, при открытом треке отправляет команду MCI. При успехе показывает и сохраняет новую громкость; при ошибке возвращает старое значение.

Побочные эффекты. Может изменить `g_volume`, громкость устройства, статус и служебное хранилище настроек.

Ошибки и особые условия. Ошибка MCI отображается пользователю, а изменение громкости откатывается.

Основные связанные вызовы. `clear_panel`, `clamp_volume`, `apply_volume_to_open_track`, `print_mci_error`, `print_volume`, `save_settings`.

4.4.23. Функция `increase_volume`

Сигнатура

```c
static void increase_volume(void);
```

Назначение. Увеличивает громкость на один шаг.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Передает положительный `VOLUME_STEP` в общую функцию изменения громкости.

Побочные эффекты. Совпадают с побочными эффектами `change_volume`.

Ошибки и особые условия. Обработка ошибок делегирована `change_volume`.

Основные связанные вызовы. `change_volume`.

4.4.24. Функция `decrease_volume`

Сигнатура

```
static void decrease_volume(void);
```

Назначение. Уменьшает громкость на один шаг.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Передает отрицательный `VOLUME_STEP` в общую функцию изменения громкости.

Побочные эффекты. Совпадают с побочными эффектами `change_volume`.

Ошибки и особые условия. Обработка ошибок делегирована `change_volume`.

Основные связанные вызовы. `change_volume`.

4.5. Воспроизведение и MCI

4.5.25. Функция `print_mci_error`

Сигнатура

```
static void print_mci_error(const char* message, MCIERROR err);
```

Назначение. Преобразует код ошибки MCI в понятное статусное сообщение.

Параметры. `message` - контекст операции; `err` - код ошибки `MCIERROR`.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Запрашивает wide-текст ошибки у MCI, переводит его в UTF-8 и записывает в статус. Если текст получить нельзя, выводит числовой код.

Побочные эффекты. Перезаписывает глобальную строку статуса.

Ошибки и особые условия. Предусмотрен резервный вывод числового кода при ошибке получения или преобразования текста.

Основные связанные вызовы. `mciGetErrorStringW`, `WideCharToMultiByte`, `set_status`.

4.5.26. Функция `get_current_track_duration_ms`

Сигнатура

```
static int get_current_track_duration_ms(long* duration_ms);
```

Назначение. Получает длительность открытого трека в миллисекундах.

Параметры. `duration_ms` - выходной указатель на значение длительности.

Возвращаемое значение. `1` при успешном чтении неотрицательной длительности, иначе `0`.

Алгоритм работы. Устанавливает миллисекундный формат времени MCI, запрашивает длину трека и преобразует полученную строку в число через `strtol`.

Побочные эффекты. Записывает длительность по адресу `duration_ms`; меняет формат времени MCI.

Ошибки и особые условия. Возвращает `0` при `NULL`, ошибке MCI или невозможности разобрать числовой ответ.

Основные связанные вызовы. `mciSendStringA`, `strtol`.

4.5.27. Функция `format_duration`

Сигнатура

```
static void format_duration(long duration_ms, char* output, size_t
output_size);
```

Назначение. Форматирует длительность трека в виде `мм:сс`.

Параметры. `duration_ms` - длительность в миллисекундах; `output` - выходной буфер; `output_size` - размер буфера.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Для отрицательного значения записывает `--:--`. Иначе переводит миллисекунды в минуты и секунды и формирует строку с ведущими нулями.

Побочные эффекты. Изменяет `output`.

Ошибки и особые условия. При нулевом размере буфера ничего не записывает. При малом буфере результат усекается `snprintf`.

Основные связанные вызовы. `snprintf`.

4.5.28. Функция `stop_music`

Сигнатура

```
void stop_music();
```

Назначение. Останавливает и закрывает текущее MCI-устройство.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Отправляет команды `stop track` и `close track`, затем сбрасывает флаги открытого трека, паузы и времени запуска.

Побочные эффекты. Останавливает звук и изменяет глобальные флаги.

Ошибки и особые условия. Коды ошибок MCI намеренно игнорируются: функция используется как безопасная очистка состояния.

Основные связанные вызовы. `mciSendStringA`.

4.5.29. Функция `play_current`

Сигнатура

```
void play_current(Playlist* pl);
```

Назначение. Открывает и запускает текущий MP3-трек плейлиста.

Параметры. `pl` - плейлист с установленным указателем `current`.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет наличие трека, расширение и существование файла. Закрывает предыдущий трек, формирует Unicode MCI-команду открытия, применяет громкость, получает длительность и запускает воспроизведение. Обновляет статус строкой с исполнителем, названием и временем.

Побочные эффекты. Управляет MCI-устройством, меняет глобальные флаги и статус.

Ошибки и особые условия. Обрабатывает пустой плейлист, неподдерживаемое расширение, отсутствие файла, слишком длинный путь, ошибку Unicode-преобразования и ошибки MCI.

Основные связанные вызовы. `clear_panel`, `file_exists_utf8`, `stop_music`, `MultiByteToWideChar`, `mciSendStringW`, `apply_volume_to_open_track`, `get_current_track_duration_ms`, `format_duration`, `mciSendStringA`, `GetTickCount`, `set_status`, `print_mci_error`.

4.5.30. Функция `play_next`

Сигнатура

```
void play_next(Playlist* pl);
```

Назначение. Переходит к следующему треку с циклическим возвратом к началу списка.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. При наличии текущего трека выбирает `current->next`, а для последнего узла - `head`; затем запускает выбранный трек.

Побочные эффекты. Изменяет `pl->current` и запускает воспроизведение.

Ошибки и особые условия. Для пустого плейлиста ничего не делает.

Основные связанные вызовы. `play_current`.

4.5.31. Функция `play_prev`

Сигнатура

```
void play_prev(Playlist* pl);
```

Назначение. Переходит к предыдущему треку с циклическим переходом к концу списка.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. При наличии текущего трека выбирает `current->prev`, а для первого узла - `tail`; затем запускает выбранный трек.

Побочные эффекты. Изменяет `pl->current` и запускает воспроизведение.

Ошибки и особые условия. Для пустого плейлиста ничего не делает.

Основные связанные вызовы. `play_current`.

4.5.32. Функция `toggle_pause`

Сигнатура

```
void toggle_pause();
```

Назначение. Переключает открытый трек между воспроизведением и паузой.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет наличие открытого трека. В зависимости от `g_is_paused` отправляет MCI-команду `pause track` или `resume track`, меняет флаг и статус.

Побочные эффекты. Управляет воспроизведением, меняет флаг паузы и статус.

Ошибки и особые условия. Сообщает об отсутствии трека и выводит ошибки MCI.

Основные связанные вызовы. `clear_panel`, `set_status`, `mciSendStringA`, `print_mci_error`.

4.6. Управление двусвязным списком

4.6.33. Функция `init_playlist`

Сигнатура

```
void init_playlist(Playlist* pl);
```

Назначение. Инициализирует пустой плейлист.

Параметры. `pl` - указатель на структуру `Playlist`.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Обнуляет `head`, `tail`, `current` и устанавливает размер списка в `0`.

Побочные эффекты. Изменяет структуру плейлиста.

Ошибки и особые условия. Ожидает ненулевой корректный указатель `pl`.

Основные связанные вызовы. Внешние функции не вызывает.

4.6.34. Функция `create_track`

Сигнатура

```
Track* create_track(const char* title, const char* artist, const char*
filepath);
```

Назначение. Создает динамический узел двусвязного списка треков.

Параметры. `title` - название; `artist` - исполнитель; `filepath` - путь к файлу.

Возвращаемое значение. Указатель на новый `Track` или `NULL` при ошибке выделения памяти.

Алгоритм работы. Выделяет память через `malloc`, копирует строки в поля структуры и обнуляет ссылки `next` и `prev`.

Побочные эффекты. Выделяет динамическую память; при ошибке изменяет статус.

Ошибки и особые условия. При невозможности выделить память возвращает `NULL`.

Основные связанные вызовы. `malloc`, `copy_string`, `set_status`.

4.6.35. Функция `add_track_end`

Сигнатура

```
void add_track_end(Playlist* pl, const char* title, const char* artist,
const char* filepath);
```

Назначение. Добавляет новый трек в конец плейлиста.

Параметры. `pl` - плейлист; `title`, `artist`, `filepath` - данные нового трека.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает узел. Для пустого списка назначает его одновременно головой, хвостом и текущим треком. Иначе связывает его с прежним хвостом. Увеличивает размер.

Побочные эффекты. Изменяет связи списка и выделяет память через `create_track`.

Ошибки и особые условия. Если создать узел не удалось, список остается без изменений.

Основные связанные вызовы. `create_track`.

4.6.36. Функция `clear_playlist`

Сигнатура

```
void clear_playlist(Playlist* pl);
```

Назначение. Полностью освобождает память плейлиста.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Линейно проходит список, запоминает следующий узел до освобождения текущего, затем сбрасывает указатели и размер.

Побочные эффекты. Освобождает всю память узлов и обнуляет плейлист.

Ошибки и особые условия. Ожидает ненулевой корректный указатель `pl`.

Основные связанные вызовы. `free`.

4.7. Импорт и редактирование плейлиста

4.7.37. Функция `add_track_interactive`

Сигнатура

```
void add_track_interactive(Playlist* pl);
```

Назначение. Интерактивно запрашивает данные одного трека и добавляет его в плейлист.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Очищает экран и буфер ввода, считывает исполнителя, название и путь. Удаляет кавычки, проверяет непустые значения, расширение `.mp3` и существование файла, затем добавляет трек.

Побочные эффекты. Читает `stdin`, пишет в `stdout`, может изменить плейлист и статус.

Ошибки и особые условия. Показывает сообщение при пустых полях, неверном расширении или отсутствующем файле.

Основные связанные вызовы. `system`, `getchar`, `fgets`, `strcspn`, `strip_surrounding_quotes`, `file_exists_utf8`, `add_track_end`, `clear_panel`, `set_status`.

4.7.38. Функция `remove_track_by_index`

Сигнатура

```
void remove_track_by_index(Playlist* pl, int index);
```

Назначение. Удаляет трек по пользовательскому порядковому номеру.

Параметры. `pl` - плейлист; `index` - номер от `1` до `pl->size`.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Находит узел линейным проходом. При удалении текущего трека останавливает воспроизведение и выбирает соседний узел. Затем перевязывает `prev` и `next`, обновляет границы списка, освобождает память и уменьшает размер.

Побочные эффекты. Меняет список, может остановить звук и освобождает память.

Ошибки и особые условия. Неверный номер или пустой список приводят к статусному сообщению без изменения списка.

Основные связанные вызовы. `clear_panel`, `set_status`, `stop_music`, `free`.

4.7.39. Функция `load_from_folder`

Сигнатура

```
void load_from_folder(Playlist* pl);
```

Назначение. Импортирует MP3-файлы из выбранной папки без рекурсивного обхода.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Запрашивает путь, формирует маску `*.*`, преобразует ее в Unicode и обходит элементы папки через WinAPI. Для файлов `.mp3` строит полный путь, пытается прочитать ID3v1-теги и добавляет трек. При отсутствии тегов использует имя файла и исполнителя `Unknown`.

Побочные эффекты. Читает `stdin`, пишет в `stdout`, обращается к файловой системе и дополняет плейлист.

Ошибки и особые условия. Обрабатывает пустой и слишком длинный путь, ошибку Unicode-преобразования и недоступную папку. Слишком длинные имена отдельных файлов пропускаются.

Основные связанные вызовы. `system`, `getchar`, `fgets`, `strip_surrounding_quotes`, `snprintf`, `MultiByteToWideChar`, `FindFirstFileW`, `FindNextFileW`, `WideCharToMultiByte`, `get_id3v1_tags`, `copy_string`, `add_track_end`, `FindClose`, `set_status`.

4.7.40. Функция `sort_playlist`

Сигнатура

```
void sort_playlist(Playlist* pl);
```

Назначение. Сортирует плейлист по названию трека без учета регистра.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Выполняет сортировку вставками непосредственно на связях двусвязного списка. Для каждого узла находит позицию в новом упорядоченном списке, обновляет `prev`, `next`, `head` и `tail`. Указатель `current` остается связан с тем же узлом.

Побочные эффекты. Перестраивает связи списка, панель результатов и статус.

Ошибки и особые условия. Для списка короче двух элементов выводит сообщение и не выполняет сортировку.

Основные связанные вызовы. `clear_panel`, `_stricmp`, `print_playlist`, `set_status`.

4.7.41. Функция `search_tracks`

Сигнатура

```
void search_tracks(Playlist* pl);
```

Назначение. Ищет треки по фрагменту названия или исполнителя.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Запрашивает поисковую строку, проходит список и добавляет найденные элементы в панель. Сравнение выполняется без учета регистра и поддерживает кириллицу.

Побочные эффекты. Читает `stdin`, пишет в `stdout`, изменяет панель и статус.

Ошибки и особые условия. Пустой запрос отклоняется. При отсутствии совпадений выводится отдельное сообщение.

Основные связанные вызовы. `system`, `getchar`, `fgets`, `strcspn`, `clear_panel`, `append_panel`, `contains_ignore_case_utf8`, `set_status`.

4.7.42. Функция `shuffle_playlist`

Сигнатура

```
void shuffle_playlist(Playlist* pl);
```

Назначение. Перемешивает порядок узлов плейлиста.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает временный массив указателей, заполняет его узлами, выполняет алгоритм Фишера-Йетса и заново связывает узлы. Значение `current` сохраняется, поскольку сами узлы не пересоздаются.

Побочные эффекты. Временно выделяет память, перестраивает список, меняет панель и статус.

Ошибки и особые условия. Для списка короче двух элементов и при ошибке `malloc` выводит сообщение без изменения порядка.

Основные связанные вызовы. `clear_panel`, `malloc`, `rand`, `free`, `print_playlist`, `set_status`.

4.8. Сохранение и загрузка данных

4.8.43. Функция `save_playlist`

Сигнатура

```
void save_playlist(Playlist* pl, const char* filename);
```

Назначение. Сохраняет плейлист в текстовый файл.

Параметры. `pl` - плейлист; `filename` - имя выходного файла.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Открывает файл на запись и сохраняет каждый трек отдельной строкой в формате `исполнитель|название|путь`.

Побочные эффекты. Создает или перезаписывает файл и обновляет статус.

Ошибки и особые условия. При невозможности открыть файл отображает ошибку и завершает работу.

Основные связанные вызовы. `clear_panel`, `fopen`, `fprintf`, `fclose`, `set_status`.

4.8.44. Функция `load_playlist`

Сигнатура

```
void load_playlist(Playlist* pl, const char* filename);
```

Назначение. Загружает плейлист из текстового файла.

Параметры. `pl` - плейлист; `filename` - имя входного файла.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Открывает файл, останавливает звук и очищает список. Читает строки формата `исполнитель|название|путь`, разделяет их через `strtok`, добавляет только доступные файлы и подсчитывает пропуски.

Побочные эффекты. Заменяет текущий плейлист, может остановить звук и меняет статус.

Ошибки и особые условия. При отсутствии файла сохраняет прежний плейлист. Некорректные строки и недоступные пути пропускаются.

Основные связанные вызовы. `clear_panel`, `fopen`, `stop_music`, `clear_playlist`, `fgets`, `strcspn`, `strtok`, `file_exists_utf8`, `add_track_end`, `fclose`, `set_status`.

4.9. Чтение метаданных ID3v1

4.9.45. Функция `trim_trailing_spaces`

Сигнатура

```
void trim_trailing_spaces(char* str);
```

Назначение. Удаляет пробельные символы в конце строки.

Параметры. `str` - изменяемая строка.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Двигается от конца строки к началу и заменяет пробелы, табуляцию и переводы строк нулевыми символами.

Побочные эффекты. Изменяет `str` на месте.

Ошибки и особые условия. Ожидает корректную строку с завершающим нулевым символом.

Основные связанные вызовы. `strlen`.

4.9.46. Функция `get_id3v1_tags`

Сигнатура

```
int get_id3v1_tags(const char* filepath, char* out_title, char* out_artist);
```

Назначение. Извлекает название и исполнителя из ID3v1-тега MP3-файла.

Параметры. `filepath` - UTF-8 путь; `out_title` - выходной буфер названия; `out_artist` - выходной буфер исполнителя.

Возвращаемое значение. `1` при успешном чтении непустых тегов, иначе `0`.

Алгоритм работы. Открывает файл по Unicode-пути в бинарном режиме, читает последние 128 байт, проверяет сигнатуру `TAG`, извлекает поля по 30 байт, удаляет пробелы и преобразует Windows-1251 в UTF-8.

Побочные эффекты. Читает файл и записывает выходные буферы.

Ошибки и особые условия. Возвращает `0` при ошибке пути, открытия, позиционирования, чтения, отсутствии сигнатуры, пустых полях или ошибке кодировки.

Основные связанные вызовы. `MultiByteToWideChar`, `_wfopen`, `fseek`, `ftell`, `fread`, `strncmp`, `memcpy`, `trim_trailing_spaces`, `WideCharToMultiByte`, `strlen`.

4.10. Главная функция программы

4.10.47. Функция `main`

Сигнатура

```
int main();
```

Назначение. Инициализирует приложение, выполняет цикл обработки команд и освобождает ресурсы при выходе.

Параметры. Параметры отсутствуют.

Возвращаемое значение. `0` после штатного завершения.

Алгоритм работы. Настраивает UTF-8 консоль, генератор случайных чисел и критическую секцию. Загружает настройки и плейлист, запускает фоновый поток. В цикле отображает экран, читает команду и вызывает соответствующую функцию. При выходе останавливает поток, сохраняет данные, закрывает MCI и освобождает список.

Побочные эффекты. Управляет жизненным циклом приложения, файлами, памятью, потоками и консолью.

Ошибки и особые условия. При ошибке чтения команды завершает цикл. Ошибки отдельных операций обрабатываются вызываемыми функциями.

Основные связанные вызовы. `SetConsoleOutputCP`, `SetConsoleCP`, `srand`, `InitializeCriticalSection`, `init_playlist`, `load_settings`, `file_exists_utf8`, `load_playlist`, `start_auto_next_monitor`, `render_screen`, функции меню, `stop_auto_next_monitor`, `save_playlist`, `save_settings`, `stop_music`, `clear_playlist`, `DeleteCriticalSection`.

5. Описание функций файла `tests/test_musicplayer.c`

В тестовом модуле определено функций: **15**. Макрос `TEST_ASSERT` не является функцией и описывается отдельно.

Результат штатного запуска автоматизированных тестов показан на рисунке 5.



Рисунок 5 - Результат выполнения тестов

5.1. Макрос `TEST_ASSERT`

Макрос принимает логическое условие, увеличивает счетчик выполненных проверок и при ложном результате увеличивает счетчик ошибок и печатает файл, строку и текст условия.

5.1. Вспомогательные функции тестового модуля

5.1.1. Функция `reset_ui_state`

Сигнатура

```
static void reset_ui_state(void);
```

Назначение. Возвращает панель и статус интерфейса в предсказуемое тестовое состояние.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Очищает панель и устанавливает статус `test`.

Побочные эффекты. Изменяет глобальное состояние интерфейса.

Ошибки и особые условия. Ошибочные ситуации отсутствуют.

Основные связанные вызовы. `clear_panel`, `set_status`.

5.1.2. Функция `reset_playlist`

Сигнатура

```
static void reset_playlist(Playlist* pl);
```

Назначение. Останавливает звук и сбрасывает переданный плейлист перед тестом.

Параметры. `pl` - плейлист.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Останавливает MCI, очищает список, повторно инициализирует структуру и сбрасывает интерфейс.

Побочные эффекты. Освобождает память списка и меняет глобальное состояние.

Ошибки и особые условия. Ожидает корректный указатель `pl`.

Основные связанные вызовы. `stop_music`, `clear_playlist`, `init_playlist`, `reset_ui_state`.

5.1.3. Функция `create_file`

Сигнатура

```
static void create_file(const char* path);
```

Назначение. Создает небольшой временный файл для файловых тестов.

Параметры. `path` - путь создаваемого файла.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Открывает файл в бинарном режиме, проверяет успешность через тестовый макрос, записывает строку `test` и закрывает файл.

Побочные эффекты. Создает или перезаписывает файл, увеличивает счетчики проверок.

Ошибки и особые условия. Ошибка открытия фиксируется макросом `TEST_ASSERT`.

Основные связанные вызовы. `fopen`, `TEST_ASSERT`, `fputs`, `fclose`.

5.1.4. Функция `backup_settings_file`

Сигнатура

```
static void backup_settings_file(void);
```

Назначение. Сохраняет пользовательский файл настроек перед тестами.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Удаляет старую временную копию и пытается переименовать служебное хранилище настроек в резервный файл.

Побочные эффекты. Может переименовать файл и установить флаг резервного копирования.

Ошибки и особые условия. Отсутствие исходного файла считается допустимым.

Основные связанные вызовы. `remove`, `rename`.

5.1.5. Функция `restore_settings_file`

Сигнатура

```
static void restore_settings_file(void);
```

Назначение. Восстанавливает пользовательский файл настроек после тестов.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Удаляет тестовый служебное хранилище настроек и при наличии резервной копии возвращает ей исходное имя.

Побочные эффекты. Удаляет и переименовывает файлы.

Ошибки и особые условия. Ошибки файловых операций отдельно не выводятся.

Основные связанные вызовы. `remove`, `rename`.

5.2. Тестовые сценарии

5.2.6. Функция `test_string_helpers`

Сигнатура

```
static void test_string_helpers(void);
```

Назначение. Проверяет обработку кавычек, ограниченное копирование строк и форматирование длительности.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Выполняет набор утверждений для корректных и граничных значений служебных функций.

Побочные эффекты. Увеличивает счетчики тестов.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `strip_surrounding_quotes`, `copy_string`, `format_duration`, `TEST_ASSERT`.

5.2.7. Функция `test_volume_helpers`

Сигнатура

```
static void test_volume_helpers(void);
```

Назначение. Проверяет ограничение диапазона и шаг изменения громкости.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет нижнюю границу, обычное значение, верхнюю границу, уменьшение и последующее увеличение громкости.

Побочные эффекты. Изменяет глобальную громкость и может перезаписать служебное хранилище настроек.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `clamp_volume`, `decrease_volume`, `increase_volume`, `TEST_ASSERT`.

5.2.8. Функция `test_playlist_add_remove_and_clear`

Сигнатура

```c
static void test_playlist_add_remove_and_clear(void);
```

Назначение. Проверяет добавление, удаление и очистку узлов плейлиста.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает список из трех узлов, проверяет ссылки и размер, удаляет средний узел, повторно проверяет связи и полностью очищает список.

Побочные эффекты. Выделяет и освобождает динамическую память.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `init_playlist`, `add_track_end`, `remove_track_by_index`, `clear_playlist`, `TEST_ASSERT`.

5.2.9. Функция `test_sort_playlist_keeps_current_node`

Сигнатура

```c
static void test_sort_playlist_keeps_current_node(void);
```

Назначение. Проверяет сортировку и сохранение текущего узла.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает неупорядоченный плейлист, выбирает текущий узел, сортирует и проверяет порядок, размер и неизменность адреса `current`.

Побочные эффекты. Выделяет и освобождает память списка.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `init_playlist`, `add_track_end`, `sort_playlist`, `clear_playlist`, `TEST_ASSERT`.

5.2.10. Функция `test_shuffle_playlist_preserves_nodes`

Сигнатура

```c
static void test_shuffle_playlist_preserves_nodes(void);
```

Назначение. Проверяет корректность перемешивания связного списка.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает три узла, сохраняет их адреса, фиксирует seed генератора, перемешивает список и проверяет наличие каждого узла, границы, обратные связи и сохранение `current`.

Побочные эффекты. Выделяет и освобождает память списка, меняет состояние генератора случайных чисел.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `init_playlist`, `add_track_end`, `srand`, `shuffle_playlist`, `clear_playlist`, `TEST_ASSERT`.

5.2.11. Функция `test_case_insensitive_search_helper`

Сигнатура

```c
static void test_case_insensitive_search_helper(void);
```

Назначение. Проверяет регистронезависимый поиск для латиницы и кириллицы.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Проверяет положительные случаи с разным регистром, поиск кириллицы и отрицательный случай.

Побочные эффекты. Увеличивает счетчики тестов.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `contains_ignore_case_utf8`, `TEST_ASSERT`.

5.2.12. Функция `test_file_exists_utf8`

Сигнатура

```c
static void test_file_exists_utf8(void);
```

Назначение. Проверяет определение существующего и отсутствующего файла.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает временный MP3-файл-заглушку, проверяет положительный и отрицательный случаи и удаляет файл.

Побочные эффекты. Создает и удаляет временный файл.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `create_file`, `file_exists_utf8`, `remove`, `TEST_ASSERT`.

5.2.13. Функция `test_save_and_load_playlist_skip_missing_files`

Сигнатура

```c
static void test_save_and_load_playlist_skip_missing_files(void);
```

Назначение. Проверяет сохранение, загрузку и пропуск недоступных файлов.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Создает два доступных файла и один отсутствующий путь, сохраняет три записи, загружает плейлист и проверяет, что восстановлены только доступные треки.

Побочные эффекты. Создает и удаляет временные файлы, выделяет и освобождает память.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `init_playlist`, `create_file`, `add_track_end`, `save_playlist`, `load_playlist`, `clear_playlist`, `remove`, `TEST_ASSERT`.

5.2.14. Функция `test_settings_roundtrip`

Сигнатура

```c
static void test_settings_roundtrip(void);
```

Назначение. Проверяет сохранение и повторную загрузку громкости.

Параметры. Параметры отсутствуют.

Возвращаемое значение. Не возвращает значение.

Алгоритм работы. Сохраняет тестовую громкость, заменяет значение в памяти, загружает настройки и сравнивает восстановленный результат.

Побочные эффекты. Изменяет глобальную громкость и файл настроек.

Ошибки и особые условия. Неудачные утверждения фиксируются макросом `TEST_ASSERT`.

Основные связанные вызовы. `save_settings`, `load_settings`, `remove`, `TEST_ASSERT`.

5.3. Главная функция тестового модуля

5.3.15. Функция `main`

Сигнатура

```
int main(void);
```

Назначение. Запускает все автоматизированные тесты и возвращает итоговый код завершения.

Параметры. Параметры отсутствуют.

Возвращаемое значение. `0`, если все проверки успешны; `1`, если есть хотя бы одна ошибка.

Алгоритм работы. Инициализирует критическую секцию, создает каталог тестов, сохраняет настройки пользователя, выполняет тестовые сценарии, восстанавливает состояние и печатает статистику.

Побочные эффекты. Создает временные файлы, изменяет глобальное состояние и пишет результат в консоль.

Ошибки и особые условия. Итоговый код завершения позволяет CTest определить успешность набора тестов.

Основные связанные вызовы. Вспомогательные функции тестов, `InitializeCriticalSection`, `_mkdir`, `DeleteCriticalSection`, `printf`.

6. Используемые технические средства

Программа выполняется на персональном компьютере под управлением Windows. Для вывода используется консоль. Для воспроизведения требуется звуковое устройство, поддерживаемое операционной системой. Для хранения локальных данных используется файловая система.

7. Вызов и загрузка

Входной точкой рабочей программы является функция `main` из файла `main.c`. Входной точкой тестового исполняемого файла является функция `main` из `tests/test_musicplayer.c`.

8. Входные данные

Программа принимает:

- команды меню из стандартного ввода;
- пути к отдельным MP3-файлам и папкам в кодировке UTF-8;
- сохраненные данные плейлиста со строками формата `исполнитель|название|путь`;
- сохраненное значение громкости в формате `volume=<число>`;
- бинарные данные MP3-файлов для чтения последних 128 байт ID3v1.

9. Выходные данные

Программа формирует:

- текстовые сообщения консольного интерфейса в UTF-8;
- звук через Windows Multimedia API;
- сохраненные данные плейлиста;
- сохраненное значение громкости;
- диагностический вывод тестового модуля с количеством выполненных и неуспешных проверок.

10. Файлы кода

Файл	Назначение
`main.c`	Исходный код консольного плеера
`tests/test_musicplayer.c`	Автоматизированные тесты

11. Контроль полноты описания

Документированы все функции проекта: **62** из **62**.

- функции `main.c`: 47;
- функции `tests/test_musicplayer.c`: 15.

Приложение А. Полный листинг файла `main.c`

```c
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <time.h>
#include <wchar.h>
#include <windows.h> // Для работы со звуком и кодировками
#include <mmsystem.h>

#define TITLE_SIZE 100
#define ARTIST_SIZE 100
#define PATH_SIZE 512
#define COMMAND_SIZE 1024
#define PLAYLIST_LINE_SIZE (TITLE_SIZE + ARTIST_SIZE + PATH_SIZE + 4)
#define DURATION_TEXT_SIZE 16
#define MENU_INPUT_SIZE 16
#define STATUS_MESSAGE_SIZE 512
#define PANEL_TEXT_SIZE 8192
#define WIDE_TEXT_SIZE 512
#define DEFAULT_PLAYLIST_FILE "playlist.txt"
#define SETTINGS_FILE "settings.txt"
#define AUTO_NEXT_POLL_MS 1000
#define AUTO_NEXT_START_GRACE_MS 1500
#define VOLUME_MIN 0
#define VOLUME_MAX 1000
#define VOLUME_STEP 100

// --- СТРУКТУРЫ ДАННЫХ ---
typedef struct Track {
    char title[TITLE_SIZE];
    char artist[ARTIST_SIZE];
    char filepath[PATH_SIZE];

    struct Track* next;
    struct Track* prev;
} Track;

typedef struct Playlist {
    Track* head;
    Track* tail;
    Track* current;
    int size;
} Playlist;
```

```
static int g_track_open = 0;
static int g_is_paused = 0;
static int g_volume = VOLUME_MAX;
static char g_status_message[STATUS_MESSAGE_SIZE] = "Готово.";
static char g_panel_text[PANEL_TEXT_SIZE] = "";
static CRITICAL_SECTION g_player_lock;
static int g_player_lock_ready = 0;
static HANDLE g_auto_next_thread = NULL;
static volatile LONG g_auto_next_running = 0;
static DWORD g_last_track_start_tick = 0;

int get_id3v1_tags(const char* filepath, char* out_title, char* out_artist);
void play_next(Playlist* pl);
void clear_playlist(Playlist* pl);
static int clamp_volume(int volume);

static void copy_string(char* destination, size_t destination_size, const
char* source) {
    if (destination_size == 0) return;

    if (source == NULL) {
        destination[0] = '\0';
        return;
    }

    snprintf(destination, destination_size, "%s", source);
}

static void strip_surrounding_quotes(char* value) {
    size_t len = strlen(value);

    if (len >= 2 && value[0] == '"' && value[len - 1] == '"') {
        memmove(value, value + 1, len - 2);
        value[len - 2] = '\0';
    } else if (len > 0 && value[0] == '"') {
        memmove(value, value + 1, len);
    }
}

static void set_status(const char* format, ...) {
    va_list args;
    va_start(args, format);
    vsnprintf(g_status_message, sizeof(g_status_message), format, args);
    va_end(args);
}

static void clear_panel(void) {
    g_panel_text[0] = '\0';
}
```

```
static void append_panel(const char* format, ...) {
    size_t used = strlen(g_panel_text);
    size_t remaining = sizeof(g_panel_text) - used;
    va_list args;

    if (remaining <= 1) return;

    va_start(args, format);
    vsnprintf(g_panel_text + used, remaining, format, args);
    va_end(args);
}

static void lock_player(void) {
    if (g_player_lock_ready) {
        EnterCriticalSection(&g_player_lock);
    }
}

static void unlock_player(void) {
    if (g_player_lock_ready) {
        LeaveCriticalSection(&g_player_lock);
    }
}

static int utf8_to_wide(const char* source, wchar_t* destination, int
destination_size) {
    if (source == NULL || destination == NULL || destination_size <= 0)
return 0;
    return MultiByteToWideChar(CP_UTF8, 0, source, -1, destination,
destination_size) != 0;
}

static int file_exists_utf8(const char* filepath) {
    wchar_t wfilepath[PATH_SIZE];
    DWORD attributes;

    if (!utf8_to_wide(filepath, wfilepath, PATH_SIZE)) {
        return 0;
    }

    attributes = GetFileAttributesW(wfilepath);
    return attributes != INVALID_FILE_ATTRIBUTES && !(attributes &
FILE_ATTRIBUTE_DIRECTORY);
}

static int contains_ignore_case_utf8(const char* text, const char* query) {
    wchar_t wtext[WIDE_TEXT_SIZE];
    wchar_t wquery[WIDE_TEXT_SIZE];
```

```
    size_t text_len;
    size_t query_len;

    if (text == NULL || query == NULL || query[0] == '\0') return 0;

    if (!utf8_to_wide(text, wtext, WIDE_TEXT_SIZE) ||
        !utf8_to_wide(query, wquery, WIDE_TEXT_SIZE)) {
        return strstr(text, query) != NULL;
    }

    text_len = wcslen(wtext);
    query_len = wcslen(wquery);
    if (text_len == 0 || query_len == 0) return 0;

    if (query_len > text_len) return 0;

    for (size_t i = 0; i <= text_len - query_len; i++) {
        if (CompareStringOrdinal(wtext + i, (int)query_len, wquery,
(int)query_len, TRUE) == CSTR_EQUAL) {
            return 1;
        }
    }

    return 0;
}

static void save_settings(void) {
    FILE* file = fopen(SETTINGS_FILE, "w");
    if (file == NULL) return;

    fprintf(file, "volume=%d\n", g_volume);
    fclose(file);
}

static void load_settings(void) {
    FILE* file = fopen(SETTINGS_FILE, "r");
    char line[64];
    int volume;

    if (file == NULL) return;

    while (fgets(line, sizeof(line), file)) {
        if (sscanf(line, "volume=%d", &volume) == 1) {
            g_volume = clamp_volume(volume);
        }
    }

    fclose(file);
}
```

```
static void print_mci_error(const char* message, MCIERROR err) {
    wchar_t werror[256];
    char error_text[512];

    if (mciGetErrorStringW(err, werror, 256) &&
        WideCharToMultiByte(CP_UTF8, 0, werror, -1, error_text,
sizeof(error_text), NULL, NULL) != 0) {
        set_status("[!] %s: %s", message, error_text);
    } else {
        set_status("[!] %s. Код ошибки MCI: %lu", message, (unsigned
long)err);
    }
}

static int clamp_volume(int volume) {
    if (volume < VOLUME_MIN) return VOLUME_MIN;
    if (volume > VOLUME_MAX) return VOLUME_MAX;
    return volume;
}

static MCIERROR apply_volume_to_open_track(void) {
    char command[64];
    snprintf(command, sizeof(command), "setaudio track volume to %d",
g_volume);
    return mciSendStringA(command, NULL, 0, NULL);
}

static void print_volume(void) {
    clear_panel();
    set_status("[VOLUME] Громкость: %d%%", g_volume / 10);
}

static void change_volume(int delta) {
    int old_volume = g_volume;
    clear_panel();
    g_volume = clamp_volume(g_volume + delta);

    if (g_track_open) {
        MCIERROR err = apply_volume_to_open_track();
        if (err != 0) {
            g_volume = old_volume;
            print_mci_error("Ошибка изменения громкости", err);
            return;
        }
    }

    print_volume();
    save_settings();
```

```
}

static void increase_volume(void) {
    change_volume(VOLUME_STEP);
}

static void decrease_volume(void) {
    change_volume(-VOLUME_STEP);
}

static int get_current_track_duration_ms(long* duration_ms) {
    char duration_text[64];
    char* end = NULL;
    MCIERROR err;

    if (duration_ms == NULL) return 0;

    err = mciSendStringA("set track time format milliseconds", NULL, 0,
NULL);
    if (err != 0) return 0;

    duration_text[0] = '\0';
    err = mciSendStringA("status track length", duration_text,
sizeof(duration_text), NULL);
    if (err != 0) return 0;

    *duration_ms = strtol(duration_text, &end, 10);
    return end != duration_text && *duration_ms >= 0;
}

static void format_duration(long duration_ms, char* output, size_t
output_size) {
    long total_seconds;
    long minutes;
    long seconds;

    if (output_size == 0) return;

    if (duration_ms < 0) {
        snprintf(output, output_size, "--:--");
        return;
    }

    total_seconds = duration_ms / 1000;
    minutes = total_seconds / 60;
    seconds = total_seconds % 60;
    snprintf(output, output_size, "%02ld:%02ld", minutes, seconds);
}
```

```
// --- СИСТЕМНЫЕ ФУНКЦИИ СПИСКА (Блок 1) ---

// 1. Инициализация пустого плейлиста
void init_playlist(Playlist* pl) {
    pl->head = NULL;
    pl->tail = NULL;
    pl->current = NULL;
    pl->size = 0;
}

// 2. Создание нового узла
Track* create_track(const char* title, const char* artist, const char*
filepath) {
    Track* new_track = (Track*)malloc(sizeof(*new_track));
    if (new_track == NULL) {
        set_status("[!] Ошибка: не удалось выделить память для трека.");
        return NULL;
    }

    copy_string(new_track->title, sizeof(new_track->title), title);
    copy_string(new_track->artist, sizeof(new_track->artist), artist);
    copy_string(new_track->filepath, sizeof(new_track->filepath), filepath);
    new_track->next = NULL;
    new_track->prev = NULL;
    return new_track;
}

// 3. Добавление трека в конец списка
void add_track_end(Playlist* pl, const char* title, const char* artist,
const char* filepath) {
    Track* new_track = create_track(title, artist, filepath);
    if (new_track == NULL) return;

    if (pl->head == NULL) {
        pl->head = new_track;
        pl->tail = new_track;
        pl->current = new_track; // Если это первый трек, он становится
текущим
    } else {
        pl->tail->next = new_track;
        new_track->prev = pl->tail;
        pl->tail = new_track;
    }
    pl->size++;
}

// --- ФУНКЦИИ ПЛЕЕРА (Блок 2) ---

// 4. Остановка музыки
```

```
void stop_music() {
    // Останавливаем и закрываем конкретный поток track
    mciSendStringA("stop track", NULL, 0, NULL);
    mciSendStringA("close track", NULL, 0, NULL);
    g_track_open = 0;
    g_is_paused = 0;
    g_last_track_start_tick = 0;
}

// 5. Воспроизведение текущего трека (с поддержкой русских букв!)
void play_current(Playlist* pl) {
    clear_panel();

    if (pl->current == NULL) {
        set_status("[!] Плейлист пуст.");
        return;
    }

    char command[COMMAND_SIZE];
    char duration_text[DURATION_TEXT_SIZE] = "--:--";
    long duration_ms;

    const char* ext = strrchr(pl->current->filepath, '.');
    // Если это НЕ mp3, выводим красивое сообщение и выходим из функции
    if (ext == NULL || _stricmp(ext, ".mp3") != 0) {
        set_status("[!] Трек '%s' пропущен. Windows MCI поддерживает здесь
только .mp3. Нажмите '4' для следующего.", pl->current->title);
        return;
    }

    if (!file_exists_utf8(pl->current->filepath)) {
        set_status("[!] Файл трека не найден: %s", pl->current->filepath);
        return;
    }

    stop_music(); // Сначала останавливаем предыдущий трек

    // Для MP3 жестко указываем тип mpegvideo
    int written = snprintf(command, sizeof(command), "open \"%s\" type
mpegvideo alias track", pl->current->filepath);
    if (written < 0 || written >= (int)sizeof(command)) {
        set_status("[!] Ошибка: путь к файлу слишком длинный.");
        return;
    }

    // --- МАГИЯ WINDOWS: ПЕРЕВОДИМ UTF-8 в UNICODE ---
    wchar_t wcommand[COMMAND_SIZE]; // "Широкая" строка
    if (MultiByteToWideChar(CP_UTF8, 0, command, -1, wcommand, COMMAND_SIZE)
== 0) {
```

```
        set_status("[!] Ошибка преобразования пути в Unicode.");
        return;
    }

    // Вызываем функцию с буквой W (Wide), передавая ей Unicode-строку
    MCIERROR err = mciSendStringW(wcommand, NULL, 0, NULL);

    if (err != 0) {
        print_mci_error("Ошибка открытия файла", err);
        return;
    }

    g_track_open = 1;
    err = apply_volume_to_open_track();
    if (err != 0) {
        print_mci_error("Ошибка применения громкости", err);
    }

    if (get_current_track_duration_ms(&duration_ms)) {
        format_duration(duration_ms, duration_text, sizeof(duration_text));
    }

    // Запускаем воспроизведение
    err = mciSendStringA("play track", NULL, 0, NULL);
    if (err != 0) {
        print_mci_error("Ошибка воспроизведения файла", err);
        stop_music();
        return;
    }
    g_is_paused = 0;
    g_last_track_start_tick = GetTickCount();

    set_status("[▶] Играет: %s - %s (%s)", pl->current->artist, pl-
>current->title, duration_text);
}
// 6. Следующий трек (с закольцовкой)
void play_next(Playlist* pl) {
    if (pl->current == NULL) return;

    // Если есть следующий, переходим к нему. Иначе — возвращаемся в начало
(head)
    if (pl->current->next != NULL) {
        pl->current = pl->current->next;
    } else {
        pl->current = pl->head; // Плейлист начался заново
    }
    play_current(pl);
}
```

```c
// 7. Предыдущий трек (с закольцовкой)
void play_prev(Playlist* pl) {
    if (pl->current == NULL) return;

    // Если есть предыдущий, переходим к нему. Иначе — идем в самый конец
(tail)
    if (pl->current->prev != NULL) {
        pl->current = pl->current->prev;
    } else {
        pl->current = pl->tail; // Перемотали в конец
    }
    play_current(pl);
}

// 8. Красивый вывод плейлиста
void print_playlist(Playlist* pl) {
    clear_panel();

    if (pl->head == NULL) {
        set_status("[!] Плейлист пуст.");
        return;
    }

    append_panel("=== ВАШ ПЛЕЙЛИСТ (%d треков) ===\n", pl->size);
    Track* temp = pl->head;
    int index = 1;

    // Идем по списку от head до tail
    while (temp != NULL) {
        // Выделяем трек, который сейчас играет
        if (temp == pl->current) {
            append_panel(" >>> %d. %s - %s <<<\n", index, temp->artist,
temp->title);
        } else {
            append_panel("     %d. %s - %s\n", index, temp->artist, temp-
>title);
        }
        temp = temp->next;
        index++;
    }
    append_panel("================================\n");
    set_status("[INFO] Плейлист показан.");
}

// 9. Интерактивное добавление трека пользователем (с очисткой кавычек)
void add_track_interactive(Playlist* pl) {
    char title[TITLE_SIZE], artist[ARTIST_SIZE], filepath[PATH_SIZE];

    system("cls");
```

```
    printf("--- Добавление нового трека ---\n\n");

    int c;
    while ((c = getchar()) != '\n' && c != EOF); // Очистка буфера

    printf("Введите исполнителя: ");
    fgets(artist, sizeof(artist), stdin);
    artist[strcspn(artist, "\n")] = 0;

    printf("Введите название трека: ");
    fgets(title, sizeof(title), stdin);
    title[strcspn(title, "\n")] = 0;

    printf("Введите полный путь к файлу (.mp3): ");
    fgets(filepath, sizeof(filepath), stdin);
    filepath[strcspn(filepath, "\n")] = 0;

    // --- УДАЛЕНИЕ КАВЫЧЕК, если они есть ---
    strip_surrounding_quotes(filepath);

    if (artist[0] == '\0' || title[0] == '\0' || filepath[0] == '\0') {
        clear_panel();
        set_status("[!] Исполнитель, название и путь к файлу не должны быть
пустыми.");
        return;
    }

    const char* ext = strrchr(filepath, '.');
    if (ext == NULL || _stricmp(ext, ".mp3") != 0) {
        clear_panel();
        set_status("[!] Можно добавить только .mp3 файл.");
        return;
    }

    if (!file_exists_utf8(filepath)) {
        clear_panel();
        set_status("[!] Файл не найден: %s", filepath);
        return;
    }

    add_track_end(pl, title, artist, filepath);
    clear_panel();
    set_status("[+] Трек '%s - %s' успешно добавлен!", artist, title);
}
// 10. Удаление трека по номеру
void remove_track_by_index(Playlist* pl, int index) {
    clear_panel();

    if (pl->head == NULL || index < 1 || index > pl->size) {
```

```
        set_status("[!] Неверный номер трека.");
        return;
    }

    Track* current = pl->head;
    for (int i = 1; i < index; i++) {
        current = current->next;
    }

    // Если удаляем трек, который сейчас играет, останавливаем музыку
    if (current == pl->current) {
        stop_music();
        pl->current = current->next != NULL ? current->next : pl->head;
        if (pl->size == 1) pl->current = NULL; // Если это был единственный
трек
    }

    // Перевязываем указатели (классика двусвязного списка)
    if (current->prev != NULL) {
        current->prev->next = current->next;
    } else {
        pl->head = current->next; // Если удаляем первый элемент
    }

    if (current->next != NULL) {
        current->next->prev = current->prev;
    } else {
        pl->tail = current->prev; // Если удаляем последний элемент
    }

    free(current); // Освобождаем память
    pl->size--;
    set_status("[+] Трек #%d удален из плейлиста.", index);
}

// 11. Сохранение плейлиста в текстовый файл
void save_playlist(Playlist* pl, const char* filename) {
    clear_panel();

    FILE* file = fopen(filename, "w");
    if (!file) {
        set_status("[!] Ошибка: Не удалось создать файл %s", filename);
        return;
    }

    Track* temp = pl->head;
    while (temp != NULL) {
        // Записываем данные через разделитель |
        fprintf(file, "%s|%s|%s\n", temp->artist, temp->title, temp-
```

```
>filepath);
        temp = temp->next;
    }
    fclose(file);
    set_status("[+] Плейлист успешно сохранен в %s", filename);
}

// 12. Загрузка плейлиста из файла
void load_playlist(Playlist* pl, const char* filename) {
    clear_panel();

    FILE* file = fopen(filename, "r");
    if (!file) {
        set_status("[!] Ошибка: Файл %s не найден.", filename);
        return;
    }

    char line[PLAYLIST_LINE_SIZE];
    int count = 0;
    int skipped = 0;

    stop_music();
    clear_playlist(pl);

    while (fgets(line, sizeof(line), file)) {
        line[strcspn(line, "\n")] = 0; // Удаляем перенос строки

        // Разбиваем строку по разделителю |
        char* artist = strtok(line, "|");
        char* title = strtok(NULL, "|");
        char* filepath = strtok(NULL, "|");

        if (artist && title && filepath) {
            if (file_exists_utf8(filepath)) {
                add_track_end(pl, title, artist, filepath);
                count++;
            } else {
                skipped++;
            }
        }
    }
    fclose(file);
    set_status("[+] Загружено %d треков из %s. Пропущено недоступных файлов:
%d.", count, filename, skipped);
}

// 13. Полная очистка памяти (защита от утечек при выходе)
void clear_playlist(Playlist* pl) {
    Track* current = pl->head;
```

```
    while (current != NULL) {
        Track* next_track = current->next;
        free(current);
        current = next_track;
    }
    pl->head = NULL;
    pl->tail = NULL;
    pl->current = NULL;
    pl->size = 0;
}

// 14. Загрузка аудиофайлов (.mp3) из папки
void load_from_folder(Playlist* pl) {
    char folder_path[PATH_SIZE];
    int c;

    system("cls");
    printf("--- Загрузка папки ---\n\n");

    while ((c = getchar()) != '\n' && c != EOF);

    printf("Введите полный путь к папке: ");
    fgets(folder_path, sizeof(folder_path), stdin);
    folder_path[strcspn(folder_path, "\n")] = 0;

    strip_surrounding_quotes(folder_path);

    if (folder_path[0] == '\0') {
        clear_panel();
        set_status("[!] Ошибка: путь к папке не должен быть пустым.");
        return;
    }

    // ТЕПЕРЬ ИЩЕМ ВООБЩЕ ВСЕ ФАЙЛЫ (*.*)
    char search_path[COMMAND_SIZE];
    size_t len = strlen(folder_path);
    int written;
    if (len > 0 && folder_path[len - 1] != '\\' && folder_path[len - 1] !=
'/') {
        written = snprintf(search_path, sizeof(search_path), "%s\\*.*",
folder_path);
    } else {
        written = snprintf(search_path, sizeof(search_path), "%s*.*",
folder_path);
    }

    if (written < 0 || written >= (int)sizeof(search_path)) {
        clear_panel();
        set_status("[!] Ошибка: путь к папке слишком длинный.");
```

```
        return;
    }

    wchar_t wsearch_path[COMMAND_SIZE];
    if (MultiByteToWideChar(CP_UTF8, 0, search_path, -1, wsearch_path,
COMMAND_SIZE) == 0) {
        clear_panel();
        set_status("[!] Ошибка преобразования пути в Unicode.");
        return;
    }

    WIN32_FIND_DATAW findFileData;
    HANDLE hFind = FindFirstFileW(wsearch_path, &findFileData);

    if (hFind == INVALID_HANDLE_VALUE) {
        clear_panel();
        set_status("[!] Ошибка: Папка не найдена или путь неверный.");
        return;
    }

    int count = 0;
    do {
        if (!(findFileData.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY)) {
            char filename_utf8[PATH_SIZE];
            if (WideCharToMultiByte(CP_UTF8, 0, findFileData.cFileName, -1,
filename_utf8, sizeof(filename_utf8), NULL, NULL) == 0) {
                continue;
            }

            // ПРОВЕРЯЕМ РАСШИРЕНИЕ ФАЙЛА
            const char* ext = strrchr(filename_utf8, '.');
            if (ext != NULL && _stricmp(ext, ".mp3") == 0) {

                char full_filepath[PATH_SIZE];
                if (len > 0 && folder_path[len - 1] != '\\' &&
folder_path[len - 1] != '/') {
                    written = snprintf(full_filepath, sizeof(full_filepath),
"%s\\%s", folder_path, filename_utf8);
                } else {
                    written = snprintf(full_filepath, sizeof(full_filepath),
"%s%s", folder_path, filename_utf8);
                }

                if (written < 0 || written >= (int)sizeof(full_filepath)) {
                    continue;
                }

                char title[TITLE_SIZE];
                char artist[ARTIST_SIZE];
```

```
                // Для MP3 пробуем вытащить теги
                if (get_id3v1_tags(full_filepath, title, artist)) {
                    // Теги успешно загружены (ничего делать не надо,
переменные уже заполнены)
                } else {
                    // Если MP3 без тегов, берем название из файла
                    copy_string(title, sizeof(title), filename_utf8);
                    char* dot = strrchr(title, '.');
                    if (dot) *dot = '\0';
                    copy_string(artist, sizeof(artist), "Unknown");
                }

                add_track_end(pl, title, artist, full_filepath);
                count++;
            }
        }
    } while (FindNextFileW(hFind, &findFileData) != 0);

    FindClose(hFind);
    clear_panel();
    set_status("[+] Успешно загружено %d треков (.mp3)!", count);
}

// Вспомогательная функция для удаления лишних пробелов в конце строки (теги
часто забивают пробелами до 30 символов)
void trim_trailing_spaces(char* str) {
    size_t len = strlen(str);
    while (len > 0) {
        char last = str[len - 1];
        if (last != ' ' && last != '\t' && last != '\n' && last != '\r') {
            break;
        }
        str[len - 1] = '\0';
        len--;
    }
}

// 15. ПАРСЕР ТЕГОВ: Чтение бинарного файла и извлечение ID3v1
int get_id3v1_tags(const char* filepath, char* out_title, char* out_artist)
{
    // Переводим путь в Unicode, чтобы fopen смог открыть файл с русским
названием
    wchar_t wfilepath[PATH_SIZE];
    if (MultiByteToWideChar(CP_UTF8, 0, filepath, -1, wfilepath, PATH_SIZE)
== 0) {
        return 0;
    }
```

```
    // Открываем файл в режиме чтения бинарных данных ("rb" - read binary).
Используем _wfopen для Unicode путей в Windows
    FILE* file = _wfopen(wfilepath, L"rb");
    if (!file) return 0; // Файл не открылся

    if (fseek(file, 0, SEEK_END) != 0) {
        fclose(file);
        return 0;
    }

    long file_size = ftell(file);
    if (file_size < 128) {
        fclose(file);
        return 0;
    }

    // Прыгаем в самый конец файла, отступив 128 байт назад!
    if (fseek(file, -128, SEEK_END) != 0) {
        fclose(file);
        return 0;
    }

    char buffer[128];
    // Читаем эти 128 байт в наш буфер
    if (fread(buffer, 1, 128, file) != 128) {
        fclose(file);
        return 0;
    }
    fclose(file);

    // Проверяем, есть ли метка "TAG"
    if (strncmp(buffer, "TAG", 3) == 0) {
        char raw_title[31] = {0};
        char raw_artist[31] = {0};

        // Копируем 30 байт названия и 30 байт исполнителя
        memcpy(raw_title, buffer + 3, 30);
        memcpy(raw_artist, buffer + 33, 30);

        trim_trailing_spaces(raw_title);
        trim_trailing_spaces(raw_artist);

        // МАГИЯ КОДИРОВОК: Теги в MP3 в России обычно хранятся в кодировке
Windows-1251 (ANSI).
        // А наша программа работает в UTF-8. Нам нужно их перевести, иначе
будут иероглифы.
        wchar_t wtitle[256], wartist[256];
        if (MultiByteToWideChar(1251, 0, raw_title, -1, wtitle, 256) == 0 ||
            MultiByteToWideChar(1251, 0, raw_artist, -1, wartist, 256) == 0)
```

```
{
            return 0;
        }

        // Теперь из Unicode (WideChar) переводим в наш UTF-8
        if (WideCharToMultiByte(CP_UTF8, 0, wtitle, -1, out_title,
TITLE_SIZE, NULL, NULL) == 0 ||
            WideCharToMultiByte(CP_UTF8, 0, wartist, -1, out_artist,
ARTIST_SIZE, NULL, NULL) == 0) {
            return 0;
        }

        // Если теги оказались пустыми (просто пробелы), вернем 0
        if (strlen(out_title) == 0 || strlen(out_artist) == 0) return 0;

        return 1; // Успешно нашли и прочитали!
    }

    return 0; // Метки "TAG" нет
}
// 16. Пауза и продолжение
void toggle_pause() {
    clear_panel();

    if (!g_track_open) {
        set_status("[!] Сейчас нет открытого трека.");
        return;
    }

    MCIERROR err;
    if (!g_is_paused) {
        err = mciSendStringA("pause track", NULL, 0, NULL);
        if (err != 0) {
            print_mci_error("Ошибка постановки на паузу", err);
            return;
        }
        set_status("[PAUSE] Пауза");
        g_is_paused = 1;
    } else {
        err = mciSendStringA("resume track", NULL, 0, NULL);
        if (err != 0) {
            print_mci_error("Ошибка продолжения воспроизведения", err);
            return;
        }
        set_status("[PLAY] Воспроизведение продолжено");
        g_is_paused = 0;
    }
}
```

```c
// 17. Сортировка плейлиста по названию трека
void sort_playlist(Playlist* pl) {
    clear_panel();

    if (pl->head == NULL || pl->head->next == NULL) {
        set_status("[!] В плейлисте недостаточно треков для сортировки.");
        return;
    }

    Track* sorted_head = NULL;
    Track* sorted_tail = NULL;
    Track* node = pl->head;

    while (node != NULL) {
        Track* next = node->next;
        node->prev = NULL;
        node->next = NULL;

        if (sorted_head == NULL) {
            sorted_head = node;
            sorted_tail = node;
        } else if (_stricmp(node->title, sorted_head->title) < 0) {
            node->next = sorted_head;
            sorted_head->prev = node;
            sorted_head = node;
        } else {
            Track* position = sorted_head;
            while (position->next != NULL && _stricmp(position->next->title,
node->title) <= 0) {
                position = position->next;
            }

            node->next = position->next;
            node->prev = position;

            if (position->next != NULL) {
                position->next->prev = node;
            } else {
                sorted_tail = node;
            }

            position->next = node;
        }

        node = next;
    }

    pl->head = sorted_head;
    pl->tail = sorted_tail;
```

```
    print_playlist(pl);
    set_status("[+] Плейлист успешно отсортирован по алфавиту (от А до
Я)!");
}

void search_tracks(Playlist* pl) {
    char query[TITLE_SIZE];
    Track* temp;
    int index = 1;
    int found = 0;
    int c;

    system("cls");
    printf("--- Поиск треков ---\n\n");

    while ((c = getchar()) != '\n' && c != EOF);

    printf("Введите слово для поиска: ");
    fgets(query, sizeof(query), stdin);
    query[strcspn(query, "\n")] = 0;

    clear_panel();

    if (query[0] == '\0') {
        set_status("[!] Слово для поиска не должно быть пустым.");
        return;
    }

    append_panel("=== РЕЗУЛЬТАТЫ ПОИСКА: %s ===\n", query);

    temp = pl->head;
    while (temp != NULL) {
        if (contains_ignore_case_utf8(temp->title, query) ||
contains_ignore_case_utf8(temp->artist, query)) {
            append_panel("     %d. %s - %s\n", index, temp->artist, temp-
>title);
            found++;
        }

        temp = temp->next;
        index++;
    }

    if (found == 0) {
        append_panel("Совпадений не найдено.\n");
    }

    append_panel("================================\n");
```

```
    set_status("[INFO] Поиск завершен. Найдено совпадений: %d.", found);
}

void shuffle_playlist(Playlist* pl) {
    Track** tracks;
    Track* temp;
    int i;

    clear_panel();

    if (pl->size < 2) {
        set_status("[!] В плейлисте недостаточно треков для
перемешивания.");
        return;
    }

    tracks = (Track**)malloc(sizeof(*tracks) * pl->size);
    if (tracks == NULL) {
        set_status("[!] Ошибка: не удалось выделить память для
перемешивания.");
        return;
    }

    temp = pl->head;
    for (i = 0; i < pl->size; i++) {
        tracks[i] = temp;
        temp = temp->next;
    }

    for (i = pl->size - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        Track* swap = tracks[i];
        tracks[i] = tracks[j];
        tracks[j] = swap;
    }

    for (i = 0; i < pl->size; i++) {
        tracks[i]->prev = i > 0 ? tracks[i - 1] : NULL;
        tracks[i]->next = i < pl->size - 1 ? tracks[i + 1] : NULL;
    }

    pl->head = tracks[0];
    pl->tail = tracks[pl->size - 1];
    free(tracks);

    print_playlist(pl);
    set_status("[+] Плейлист перемешан.");
}
```

```
DWORD WINAPI auto_next_monitor(LPVOID parameter) {
    Playlist* pl = (Playlist*)parameter;

    while (InterlockedCompareExchange(&g_auto_next_running, 1, 1) == 1) {
        Sleep(AUTO_NEXT_POLL_MS);

        lock_player();

        if (g_track_open && !g_is_paused && g_last_track_start_tick != 0 &&
            GetTickCount() - g_last_track_start_tick >=
AUTO_NEXT_START_GRACE_MS) {
            char mode[32] = "";
            MCIERROR err = mciSendStringA("status track mode", mode,
sizeof(mode), NULL);

            if (err == 0 && strcmp(mode, "stopped") == 0 && pl->current !=
NULL) {
                set_status("[AUTO] Трек закончился. Включаю следующий.");
                play_next(pl);
            }
        }

        unlock_player();
    }

    return 0;
}

static void start_auto_next_monitor(Playlist* pl) {
    InterlockedExchange(&g_auto_next_running, 1);
    g_auto_next_thread = CreateThread(NULL, 0, auto_next_monitor, pl, 0,
NULL);
    if (g_auto_next_thread == NULL) {
        InterlockedExchange(&g_auto_next_running, 0);
        set_status("[!] Не удалось запустить автопереход на следующий
трек.");
    }
}

static void stop_auto_next_monitor(void) {
    if (g_auto_next_thread == NULL) return;

    InterlockedExchange(&g_auto_next_running, 0);
    WaitForSingleObject(g_auto_next_thread, INFINITE);
    CloseHandle(g_auto_next_thread);
    g_auto_next_thread = NULL;
}

// --- ИНТЕРФЕЙС ---
```

```c
void print_menu() {
    printf("\n[ УПРАВЛЕНИЕ ПЛЕЕРОМ ]\n");
    printf("1. Играть          | 6. Показать плейлист\n");
    printf("2. Пауза           | 7. Добавить трек / Импорт папки\n");
    printf("3. Стоп            | 8. Удалить трек\n");
    printf("4. Следующий       | 9. Отсортировать по алфавиту\n");
    printf("5. Предыдущий      | 10. Сохр. / Загр. в файл\n");
    printf("11. Поиск          | 12. Shuffle\n");
    printf("+. Громкость выше | -. Громкость ниже (сейчас %d%%)\n", g_volume
/ 10);
    printf("0. Выход\n");
    printf("Ваш выбор: ");
}

void render_screen() {
    system("cls");

    printf("[ СТАТУС ]\n");
    printf("%s\n\n", g_status_message);

    if (g_panel_text[0] != '\0') {
        printf("%s\n", g_panel_text);
    }

    print_menu();
}

#ifndef MUSICPLAYER_TEST
int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    srand((unsigned int)time(NULL));
    InitializeCriticalSection(&g_player_lock);
    g_player_lock_ready = 1;

    Playlist my_playlist;
    init_playlist(&my_playlist);
    load_settings();

    if (file_exists_utf8(DEFAULT_PLAYLIST_FILE)) {
        load_playlist(&my_playlist, DEFAULT_PLAYLIST_FILE);
    } else {
        set_status("Готово. Автосохраненный плейлист пока не найден.");
    }

    start_auto_next_monitor(&my_playlist);

    char choice[MENU_INPUT_SIZE];
    int running = 1;
```

```
    do {
        lock_player();
        render_screen();
        unlock_player();

        if (scanf("%15s", choice) != 1) break;

        lock_player();

        if (strcmp(choice, "1") == 0) {
            play_current(&my_playlist);
        } else if (strcmp(choice, "2") == 0) {
            toggle_pause();
        } else if (strcmp(choice, "3") == 0) {
            stop_music();
            clear_panel();
            set_status("[STOP] Музыка остановлена.");
        } else if (strcmp(choice, "4") == 0) {
            play_next(&my_playlist);
        } else if (strcmp(choice, "5") == 0) {
            play_prev(&my_playlist);
        } else if (strcmp(choice, "6") == 0) {
            print_playlist(&my_playlist);
        } else if (strcmp(choice, "7") == 0) {
            printf("1 - Добавить один трек, 2 - Загрузить папку: ");
            int sub7;
            if (scanf("%d", &sub7) == 1) {
                if (sub7 == 1) add_track_interactive(&my_playlist);
                else if (sub7 == 2) load_from_folder(&my_playlist);
            }
        } else if (strcmp(choice, "8") == 0) {
            print_playlist(&my_playlist);
            render_screen();
            if (my_playlist.size > 0) {
                printf("Введите номер трека для удаления: ");
                int index;
                if (scanf("%d", &index) == 1)
remove_track_by_index(&my_playlist, index);
            }
        } else if (strcmp(choice, "9") == 0) {
            sort_playlist(&my_playlist);
        } else if (strcmp(choice, "10") == 0) {
            printf("1 - Сохранить, 2 - Загрузить: ");
            int sub10;
            if (scanf("%d", &sub10) == 1) {
                if (sub10 == 1) save_playlist(&my_playlist,
DEFAULT_PLAYLIST_FILE);
                else if (sub10 == 2) load_playlist(&my_playlist,
```

```
DEFAULT_PLAYLIST_FILE);
            }
        } else if (strcmp(choice, "11") == 0) {
            search_tracks(&my_playlist);
        } else if (strcmp(choice, "12") == 0) {
            shuffle_playlist(&my_playlist);
        } else if (strcmp(choice, "+") == 0) {
            increase_volume();
        } else if (strcmp(choice, "-") == 0) {
            decrease_volume();
        } else if (strcmp(choice, "0") == 0) {
            running = 0;
        } else {
            clear_panel();
            set_status("[!] Неверный выбор.");
        }

        unlock_player();
    } while (running);

    stop_auto_next_monitor();

    lock_player();
    save_playlist(&my_playlist, DEFAULT_PLAYLIST_FILE);
    save_settings();
    stop_music();
    clear_playlist(&my_playlist);
    unlock_player();

    if (g_player_lock_ready) {
        DeleteCriticalSection(&g_player_lock);
        g_player_lock_ready = 0;
    }

    system("cls");
    printf("Выход...\n");

    return 0;
}
#endif
```

Приложение Б. Полный листинг файла

tests/test_musicplayer.c

```
  #define MUSICPLAYER_TEST
  #include "../main.c"

  #include <direct.h>

  static int tests_run = 0;
  static int tests_failed = 0;
  static int settings_was_backed_up = 0;
  static const char* settings_backup_path = "tests\\settings_backup.tmp";

  #define TEST_ASSERT(condition) do { \
      tests_run++; \
      if (!(condition)) { \
          tests_failed++; \
          printf("FAIL: %s:%d: %s\n", __FILE__, __LINE__, #condition); \
      } \
  } while (0)

  static void reset_ui_state(void) {
      clear_panel();
      set_status("test");
  }

  static void reset_playlist(Playlist* pl) {
      stop_music();
      clear_playlist(pl);
      init_playlist(pl);
      reset_ui_state();
  }

  static void create_file(const char* path) {
      FILE* file = fopen(path, "wb");
      TEST_ASSERT(file != NULL);
      if (file != NULL) {
          fputs("test", file);
          fclose(file);
      }
  }

  static void backup_settings_file(void) {
      remove(settings_backup_path);
      if (rename(SETTINGS_FILE, settings_backup_path) == 0) {
          settings_was_backed_up = 1;
      }
  }
```

```
static void restore_settings_file(void) {
    remove(SETTINGS_FILE);
    if (settings_was_backed_up) {
        rename(settings_backup_path, SETTINGS_FILE);
        settings_was_backed_up = 0;
    }
}

static void test_string_helpers(void) {
    char value1[PATH_SIZE] = "\"C:\\Music\\song.mp3\"";
    char value2[PATH_SIZE] = "\"C:\\Music\\song.mp3";
    char value3[PATH_SIZE] = "C:\\Music\\song.mp3";
    char copied[8];
    char duration[DURATION_TEXT_SIZE];

    strip_surrounding_quotes(value1);
    TEST_ASSERT(strcmp(value1, "C:\\Music\\song.mp3") == 0);

    strip_surrounding_quotes(value2);
    TEST_ASSERT(strcmp(value2, "C:\\Music\\song.mp3") == 0);

    strip_surrounding_quotes(value3);
    TEST_ASSERT(strcmp(value3, "C:\\Music\\song.mp3") == 0);

    copy_string(copied, sizeof(copied), "123456789");
    TEST_ASSERT(strcmp(copied, "1234567") == 0);

    format_duration(225000, duration, sizeof(duration));
    TEST_ASSERT(strcmp(duration, "03:45") == 0);

    format_duration(-1, duration, sizeof(duration));
    TEST_ASSERT(strcmp(duration, "--:--") == 0);
}

static void test_volume_helpers(void) {
    g_track_open = 0;
    g_volume = VOLUME_MAX;

    TEST_ASSERT(clamp_volume(-50) == VOLUME_MIN);
    TEST_ASSERT(clamp_volume(500) == 500);
    TEST_ASSERT(clamp_volume(2000) == VOLUME_MAX);

    decrease_volume();
    TEST_ASSERT(g_volume == 900);

    increase_volume();
    TEST_ASSERT(g_volume == 1000);
}
```

```
static void test_playlist_add_remove_and_clear(void) {
    Playlist pl;
    init_playlist(&pl);

    add_track_end(&pl, "Gamma", "Artist C", "c.mp3");
    add_track_end(&pl, "Alpha", "Artist A", "a.mp3");
    add_track_end(&pl, "Beta", "Artist B", "b.mp3");

    TEST_ASSERT(pl.size == 3);
    TEST_ASSERT(pl.head != NULL);
    TEST_ASSERT(pl.tail != NULL);
    TEST_ASSERT(pl.current == pl.head);
    TEST_ASSERT(strcmp(pl.head->title, "Gamma") == 0);
    TEST_ASSERT(strcmp(pl.tail->title, "Beta") == 0);
    TEST_ASSERT(pl.head->next->prev == pl.head);

    remove_track_by_index(&pl, 2);
    TEST_ASSERT(pl.size == 2);
    TEST_ASSERT(strcmp(pl.head->title, "Gamma") == 0);
    TEST_ASSERT(strcmp(pl.tail->title, "Beta") == 0);
    TEST_ASSERT(pl.head->next == pl.tail);
    TEST_ASSERT(pl.tail->prev == pl.head);

    clear_playlist(&pl);
    TEST_ASSERT(pl.size == 0);
    TEST_ASSERT(pl.head == NULL);
    TEST_ASSERT(pl.tail == NULL);
    TEST_ASSERT(pl.current == NULL);
}

static void test_sort_playlist_keeps_current_node(void) {
    Playlist pl;
    Track* original_current;
    init_playlist(&pl);

    add_track_end(&pl, "Gamma", "Artist C", "c.mp3");
    add_track_end(&pl, "Alpha", "Artist A", "a.mp3");
    add_track_end(&pl, "Beta", "Artist B", "b.mp3");
    pl.current = pl.tail;
    original_current = pl.current;

    sort_playlist(&pl);

    TEST_ASSERT(pl.size == 3);
    TEST_ASSERT(strcmp(pl.head->title, "Alpha") == 0);
    TEST_ASSERT(strcmp(pl.head->next->title, "Beta") == 0);
    TEST_ASSERT(strcmp(pl.tail->title, "Gamma") == 0);
    TEST_ASSERT(pl.current == original_current);
```

```
    TEST_ASSERT(strcmp(pl.current->title, "Beta") == 0);

    clear_playlist(&pl);
}

static void test_shuffle_playlist_preserves_nodes(void) {
    Playlist pl;
    Track* first;
    Track* second;
    Track* third;
    int seen_first = 0;
    int seen_second = 0;
    int seen_third = 0;
    Track* node;
    int count = 0;

    init_playlist(&pl);
    add_track_end(&pl, "One", "Artist", "1.mp3");
    add_track_end(&pl, "Two", "Artist", "2.mp3");
    add_track_end(&pl, "Three", "Artist", "3.mp3");

    first = pl.head;
    second = pl.head->next;
    third = pl.tail;
    pl.current = second;

    srand(1);
    shuffle_playlist(&pl);

    TEST_ASSERT(pl.size == 3);
    TEST_ASSERT(pl.current == second);
    TEST_ASSERT(pl.head->prev == NULL);
    TEST_ASSERT(pl.tail->next == NULL);

    for (node = pl.head; node != NULL; node = node->next) {
        if (node == first) seen_first = 1;
        if (node == second) seen_second = 1;
        if (node == third) seen_third = 1;
        if (node->next != NULL) TEST_ASSERT(node->next->prev == node);
        count++;
    }

    TEST_ASSERT(count == 3);
    TEST_ASSERT(seen_first && seen_second && seen_third);

    clear_playlist(&pl);
}

static void test_case_insensitive_search_helper(void) {
```

```
    TEST_ASSERT(contains_ignore_case_utf8("Night Song", "night"));
    TEST_ASSERT(contains_ignore_case_utf8("Artist Name", "NAME"));
    TEST_ASSERT(contains_ignore_case_utf8("Ночная песня", "ноч"));
    TEST_ASSERT(!contains_ignore_case_utf8("Artist Name", "album"));
}

static void test_file_exists_utf8(void) {
    create_file("tests\\exists_test.mp3");
    TEST_ASSERT(file_exists_utf8("tests\\exists_test.mp3"));
    TEST_ASSERT(!file_exists_utf8("tests\\missing_test.mp3"));
    remove("tests\\exists_test.mp3");
}

static void test_save_and_load_playlist_skip_missing_files(void) {
    Playlist source;
    Playlist loaded;

    init_playlist(&source);
    init_playlist(&loaded);

    create_file("tests\\load_ok_1.mp3");
    create_file("tests\\load_ok_2.mp3");

    add_track_end(&source, "First", "Artist 1", "tests\\load_ok_1.mp3");
    add_track_end(&source, "Missing", "Artist 2", "tests\\
missing_song.mp3");
    add_track_end(&source, "Second", "Artist 3", "tests\\load_ok_2.mp3");

    save_playlist(&source, "tests\\playlist_test.txt");
    load_playlist(&loaded, "tests\\playlist_test.txt");

    TEST_ASSERT(loaded.size == 2);
    TEST_ASSERT(strcmp(loaded.head->title, "First") == 0);
    TEST_ASSERT(strcmp(loaded.tail->title, "Second") == 0);

    clear_playlist(&source);
    clear_playlist(&loaded);
    remove("tests\\playlist_test.txt");
    remove("tests\\load_ok_1.mp3");
    remove("tests\\load_ok_2.mp3");
}

static void test_settings_roundtrip(void) {
    int previous_volume = g_volume;

    g_volume = 300;
    save_settings();

    g_volume = 1000;
```

```
    load_settings();
    TEST_ASSERT(g_volume == 300);

    g_volume = previous_volume;
    remove(SETTINGS_FILE);
}

int main(void) {
    InitializeCriticalSection(&g_player_lock);
    g_player_lock_ready = 1;
    _mkdir("tests");
    backup_settings_file();

    reset_ui_state();

    test_string_helpers();
    test_volume_helpers();
    test_playlist_add_remove_and_clear();
    test_sort_playlist_keeps_current_node();
    test_shuffle_playlist_preserves_nodes();
    test_case_insensitive_search_helper();
    test_file_exists_utf8();
    test_save_and_load_playlist_skip_missing_files();
    test_settings_roundtrip();

    stop_music();
    restore_settings_file();
    DeleteCriticalSection(&g_player_lock);
    g_player_lock_ready = 0;

    printf("Tests run: %d\n", tests_run);
    printf("Tests failed: %d\n", tests_failed);

    return tests_failed == 0 ? 0 : 1;
}
```